TP 01 — Jeu de la vie de Conway

Inside ESILV : une journée dans la peau d'un élève-ingénieur

Sixtine Michel

De Vinci Higher Education — ESILV

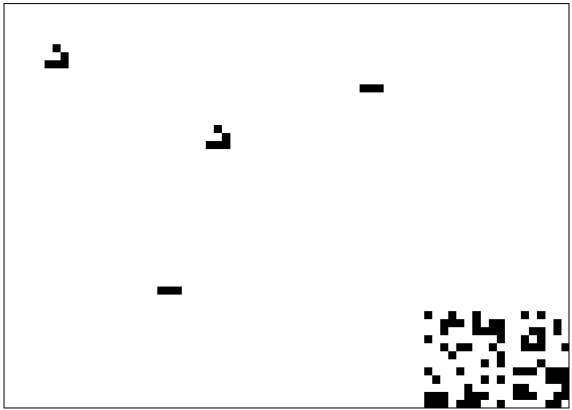
ESILV
DE VINCI
ENGINEERING
SCHOOL

Notre mission

Objectif

Faire évoluer une population de cellules à partir de quatre règles locales très simples.

Chaque cellule observe uniquement ses huit voisines. Pourtant, des motifs stables, oscillants ou mobiles apparaissent.



D'un point de vue algorithmique

Le programme sépare la création, le calcul et la mesure :

```
def creer_grille_aleatoire(nb_lignes, nb_colonnes,
                           probabilite_vie):
    # Creer le monde initial.
def compter_voisines_vivantes(grille, ligne, colonne):
    # Observer les huit cases voisines.
def calculer_generation_suivante(grille):
    # Appliquer les regles a toutes les cellules.
def compter_cellules_vivantes(grille):
    # Mesurer la population totale.
```

L'animation répète ensuite le calcul d'une génération.

Une grille de 0 et de 1

La grille est un tableau NumPy à deux dimensions :

`grille[ligne, colonne]` $\in \{0, 1\}$.

- 0 : cellule morte, affichée en blanc ;
- 1 : cellule vivante, affichée en noir.

Exemple

```
0 0 1 0 0
0 0 0 1 0
0 1 1 1 0
```

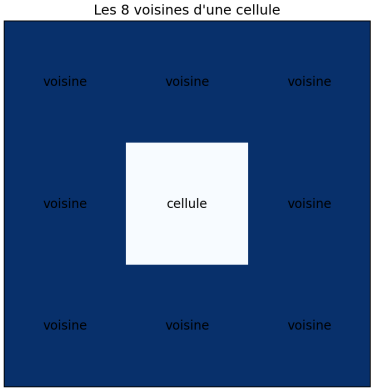
Ce petit motif de cinq cellules est un planeur.

Observer les huit voisines

Pour une cellule (l, c), les décalages de ligne et de colonne valent chacun -1, 0 ou 1.

Attention

Le couple $(0, 0)$ désigne la cellule centrale : il faut l'exclure.



Un monde qui reboucle sur lui-même

La grille est considérée comme torique : sortir d'un côté fait revenir du côté opposé. Le modulo ramène toujours les coordonnées dans la grille :

$$l_v = (l + \Delta l) \bmod N_l, \qquad c_v = (c + \Delta c) \bmod N_c.$$

Ainsi, une cellule du bord possède elle aussi huit voisines.

Les quatre règles de Conway

1. une cellule vivante avec moins de 2 voisines meurt d'isolement ;
2. avec 2 ou 3 voisines, elle survit ;
3. avec plus de 3 voisines, elle meurt de surpopulation ;
4. une cellule morte avec exactement 3 voisines naît.

Version compacte

Vivante : elle survit si $n \in \{2, 3\}$. Morte : elle naît si $n = 3$.

Les cinq étapes du programme

1. créer une grille aléatoire de 0 et de 1 ;
2. parcourir les huit voisines d'une cellule ;
3. gérer les bords et compter les voisines vivantes ;
4. appliquer les règles à toutes les cellules ;
5. compter la population et afficher son évolution.

Correspondance entre étapes et TODO

1. Grille aléatoire	**TODO 1 et 2**
2. Parcours du voisinage	**TODO 3, 4 et 5**
3. Coordonnées et compteur	**TODO 6 et 7**
4. Génération suivante	**TODO 8, 9 et 10**
5. Population totale	**TODO 11**

Créer les valeurs aléatoires — TODO 1

À vous

- Tirer une valeur aleatoire entre 0 et 1 pour chaque case.

Créer les valeurs aléatoires — TODO 1

```
valeurs_aleatoires = np.random.random(
    (nb_lignes, nb_colonnes)
)
```

NumPy produit directement un tableau de la bonne forme. Chaque valeur est tirée uniformément entre 0 et 1.

Transformer en cellules — TODO 2

À vous

- Comparer chaque valeur a la probabilite de vie.

Transformer en cellules — TODO 2

```
grille = (
    valeurs_aleatoires < probabilite_vie
).astype(int)
```

La comparaison produit des booléens. `astype(int)` transforme `False` en 0 et `True` en 1.

Parcourir le voisinage — TODO 3 et 4

À vous

- Parcourir les trois decalages de ligne : -1, 0 et 1.
- Parcourir les trois decalages de colonne : -1, 0 et 1.

Parcourir le voisinage — TODO 3 et 4

```python
for decalage_ligne in (-1, 0, 1):
    for decalage_colonne in (-1, 0, 1):
        # traiter une case du voisinage
```

Les deux boucles fabriquent les neuf couples possibles. L'étape suivante retirera le couple central.

Exclure le centre — TODO 5

À vous

- Exclure la cellule centrale de son propre voisinage.

Exclure le centre — TODO 5

```
if (decalage_ligne == 0
        and decalage_colonne == 0):
    continue
```

`continue` abandonne uniquement cette itération, puis la boucle passe au couple de décalages suivant.

Reboucler aux bords — TODO 6

À vous

- Calculer les coordonnees de la voisine avec le modulo.

Reboucler aux bords — TODO 6

```
ligne_voisine = (
    ligne + decalage_ligne) % nb_lignes
colonne_voisine = (
    colonne + decalage_colonne) % nb_colonnes
```

Par exemple, sur 50 lignes, $(-1) \bmod 50 = 49$: la voisine située au-dessus de la première ligne se trouve sur la dernière.

Compter les vivantes — TODO 7

À vous

- Ajouter l'etat 0 ou 1 de la voisine au compteur.

Compter les vivantes — TODO 7

```
compteur = compteur + grille[
    ligne_voisine, colonne_voisine
]
```

Ajouter une cellule morte ajoute 0 ; ajouter une cellule vivante ajoute 1. Après huit voisines, le compteur est compris entre 0 et 8.

Préparer le nouvel état — TODO 8

À vous

- Creer une grille de zeros ayant la meme forme et le meme type.

Préparer le nouvel état — TODO 8

```
1 nouvelle_grille = np.zeros_like(grille)
```

Il faut écrire dans une nouvelle grille. Modifier directement l'ancienne changerait les voisines avant la fin du calcul de la génération.

Faire survivre une cellule — TODO 9

À vous

- Une cellule vivante survit avec 2 ou 3 voisines.

Faire survivre une cellule — TODO 9

```
1 if nb_voisines in (2, 3):
2     nouvelle_grille[ligne, colonne] = 1
```

Cette condition est évaluée uniquement dans le cas où la cellule courante est déjà vivante.

Faire naître une cellule — TODO 10

À vous

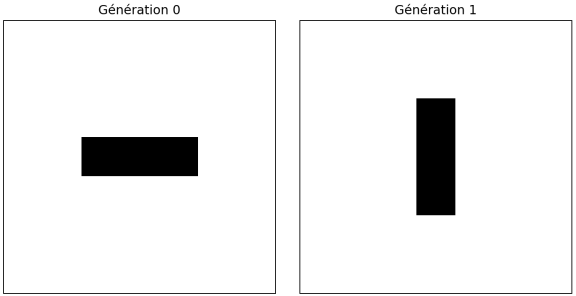
- Une cellule morte nait avec exactement 3 voisines.

Faire naître une cellule — TODO 10

```
1 if nb_voisines == 3:
2     nouvelle_grille[ligne, colonne] = 1
```

Cette seconde condition concerne uniquement une cellule actuellement morte. Toutes les autres cases restent à 0.

Un résultat intermédiaire : l'oscillateur



Trois cellules horizontales deviennent trois cellules verticales. À la génération suivante, elles reprennent leur forme initiale.

Test utile

Si ce cycle ne fonctionne pas, vérifier le comptage des voisines et les deux conditions de naissance et de survie.

Compter toute la population — TODO 11

À vous

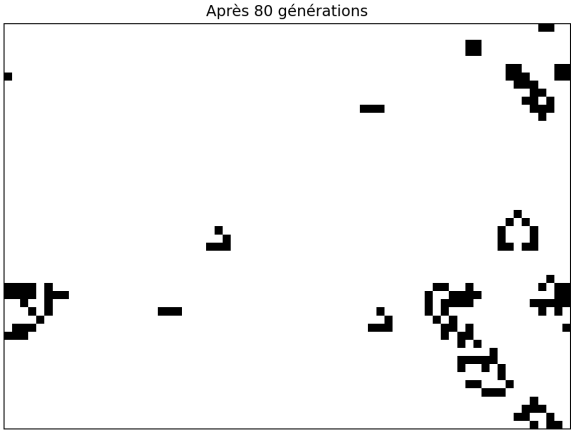
- Sommer les 1 de la grille puis convertir le resultat en int.

Compter toute la population — TODO 11

```python
return int(np.sum(grille))
```

Comme chaque cellule vivante vaut 1, la somme de toutes les cases donne directement le nombre de cellules vivantes.

Après 80 générations

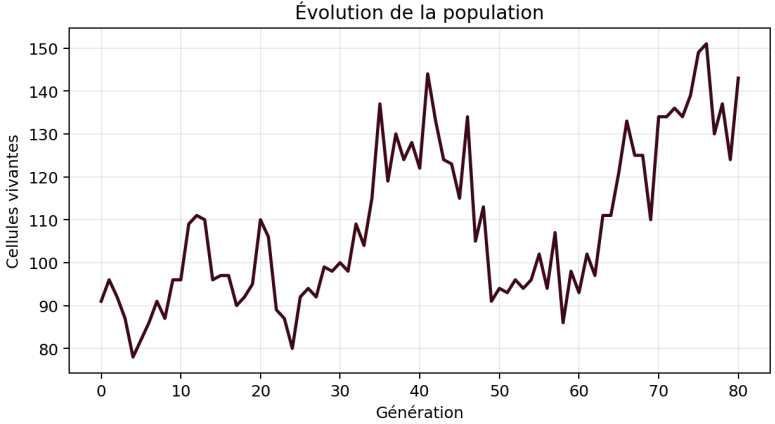


La population initiale se réorganise : certaines structures disparaissent, d'autres se stabilisent ou se déplacent.

À observer

Le planeur se déplace en diagonale sans qu'aucune direction ne lui ait été explicitement donnée.

Suivre la population



À vous d'expérimenter

- modifier `PROBABILITE_VIE` : que devient la population ?
- accélérer ou ralentir `INTERVALLE_ANIMATION` ;
- ajouter un planeur ou un oscillateur dans `creer_grille_demo` ;
- appuyer sur **R** pour créer une nouvelle grille aléatoire ;
- utiliser la barre d'espace pour mettre l'animation en pause.

À retenir

- tableau NumPy 2D ;
- indices ligne et colonne ;
- boucles imbriquées ;
- voisinage et modulo ;
- conditions et `continue` ;
- nouvelle grille indépendante ;
- somme d'un tableau ;
- comportement émergent.

Des règles locales peuvent créer des formes complexes !